Motor Insurance Management System

ABSTRACT

The **Motor Insurance Management System** is a database-driven application developed to automate and simplify the process of managing motor insurance policies. The system provides an efficient platform for maintaining customer information, vehicle details, insurance quotes, premium calculations, policy issuance, payment records, and claim management.

The main objective of this project is to replace the traditional manual insurance process with a computerized system that improves accuracy, security, and efficiency. The system enables users to register customer details, record vehicle information, generate insurance quotes based on selected coverage, calculate premiums with applicable taxes, process payments, and automatically generate insurance policies. It also maintains claim records and policy status for future reference.

The project is implemented using **MySQL** as the backend database and SQL concepts such as **Primary Key, Foreign Key, Joins, Views, Stored Procedures, Triggers, and Aggregate Functions** to ensure data integrity and efficient data management. The database is designed with multiple related tables including Users, Customers, Brokers, Vehicle Make, Vehicle Model, Vehicle Details, Quote Details, Premium Rate, Policy, Payment, and Claim Details.

This system reduces paperwork, minimizes human errors, improves data consistency, and provides quick access to insurance information. It also enhances customer service by enabling faster policy generation and claim processing. The Motor Insurance Management System is suitable for insurance companies, brokers, and agents to efficiently manage their day-to-day insurance operations and can be further enhanced with online payment gateways, SMS/email notifications, and mobile application support.

1. Introduction

The **Motor Insurance Management System** is a database-driven application developed to automate the motor insurance process. It helps insurance companies manage customer details, vehicle information, premium calculation, quote generation, policy issuance, payment tracking, and claim management efficiently.

The system reduces paperwork, improves data accuracy, and provides faster service to customers through a centralized database.

2. Problem Statement

Traditional motor insurance management is time-consuming and involves manual paperwork, which may lead to data redundancy and errors. The proposed system automates all insurance operations using a relational database, making the process faster, more secure, and reliable.

3. Objectives

- To maintain customer information.
- To store vehicle details.
- To generate insurance quotes.
- To calculate insurance premiums.
- To issue insurance policies.
- To manage policy payments.
- To maintain claim records.
- To generate reports for management.

4. Scope of the Project

This project can be used by:

- Insurance Companies
- Brokers
- Sales Agents
- Customers
- Insurance Administrators

The system supports the complete insurance process from customer registration to policy generation and claim management.

5. Modules

Module 1 – User Management

- User Registration
- Login Authentication
- User Roles (Admin, Broker, Customer)

Module 2 – Customer Management

- Add Customer
- Update Customer
- Delete Customer
- View Customer Details

Module 3 – Vehicle Management

- Vehicle Make
- Vehicle Model
- Vehicle Registration
- Vehicle Information

Module 4 – Quote Management

- Generate Quote
- Select Insurance Cover
- Premium Calculation

Module 5 – Policy Management

- Policy Generation
- Policy Issue
- Policy Renewal
- Policy Expiry

Module 6 – Payment Management

- Premium Payment
- Payment Status
- Payment History

Module 7 – Claim Management

- Register Claim
- Claim Verification
- Claim Approval
- Claim Rejection

7. Software Requirements

Software	Description
Operating System	Windows 10 / Windows 11
Database	MySQL 8.0
IDE	MySQL Workbench
Front End	Visual Studio / Java / HTML
Language	SQL

7. Hardware Requirements

- Processor : Intel Core i3 or Above
- RAM : 4 GB or Above
- Hard Disk : 100 GB
- Monitor : 15-inch Display

8. Database Tables

1. users
2. login
3. broker
4. customer
5. vehicle_make
6. vehicle_model
7. vehicle

8. premium_rate
9. quote_details
10. policy
11. payment
12. claim_details

9. Database Relationships

- Vehicle Make → Vehicle Model (One-to-Many)
- Customer → Vehicle (One-to-Many)
- Customer → Quote (One-to-Many)
- Vehicle → Quote (One-to-One)
- Quote → Policy (One-to-One)
- Policy → Payment (One-to-Many)
- Policy → Claim (One-to-Many)

10. SQL Concepts Used

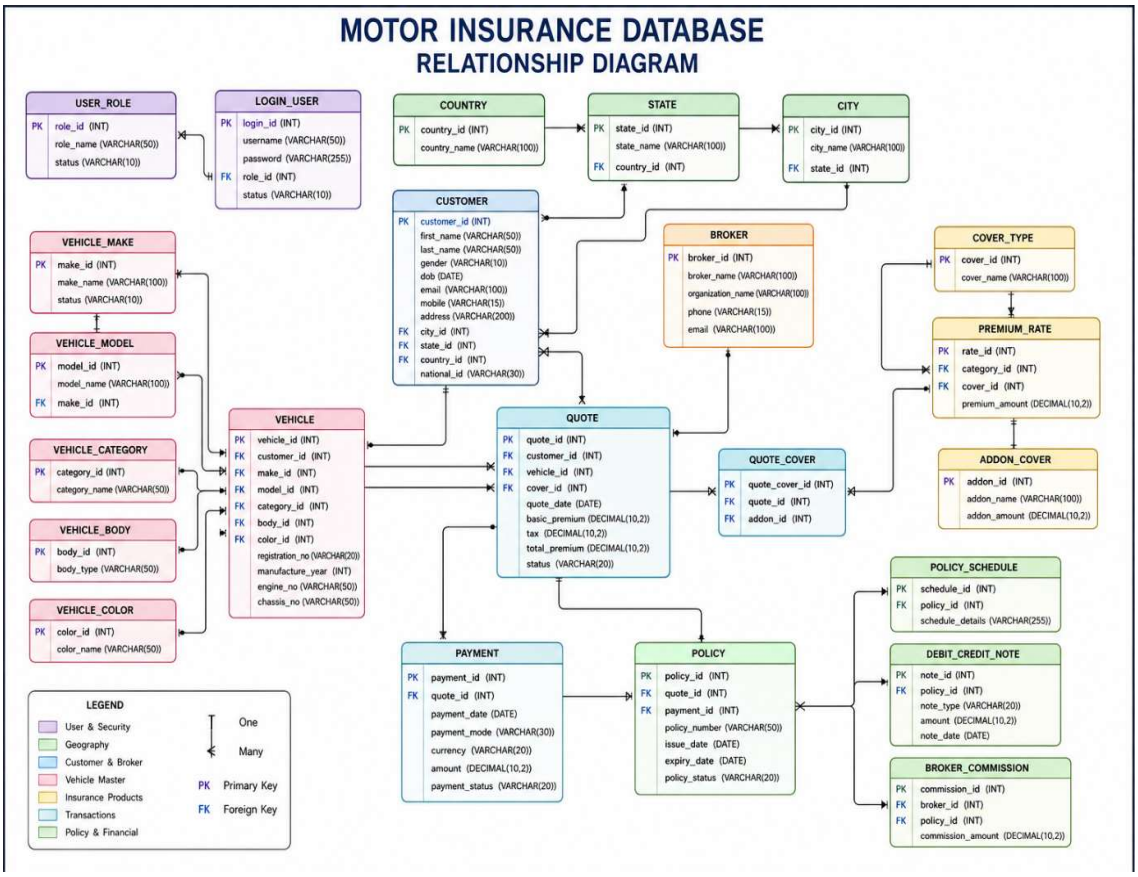
- CREATE DATABASE
- CREATE TABLE
- PRIMARY KEY
- FOREIGN KEY
- INSERT
- UPDATE
- DELETE
- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING
- INNER JOIN

- LEFT JOIN
- RIGHT JOIN
- VIEW
- STORED PROCEDURE
- TRIGGER
- AGGREGATE FUNCTIONS

11. System Workflow

1. User logs into the system.
2. Customer details are entered.
3. Vehicle information is registered.
4. Insurance cover type is selected.
5. Premium is calculated.
6. Quote is generated.
7. Customer completes the payment.
8. Policy is generated automatically.
9. Payment details are stored.
10. Claims can be submitted if required.

FLOW CHART:


MOTOR INSURANCE DATABASE
RELATIONSHIP DIAGRAM
USER_ROLE
PK role_id (INT)
role_name (VARCHAR(50))
status (VARCHAR(10))
LOGIN_USER
PK login_id (INT)
username (VARCHAR(50))
password (VARCHAR(255))
FK role_id (INT)
status (VARCHAR(10))
COUNTRY
PK country_id (INT)
country_name (VARCHAR(100))
STATE
PK state_id (INT)
state_name (VARCHAR(100))
FK country_id (INT)
CITY
PK city_id (INT)
city_name (VARCHAR(100))
FK state_id (INT)
CUSTOMER
PK customer_id (INT)
first_name (VARCHAR(50))
last_name (VARCHAR(50))
gender (VARCHAR(10))
dob (DATE)
email (VARCHAR(100))
mobile (VARCHAR(15))
address (VARCHAR(200))
FK city_id (INT)
FK state_id (INT)
FK country_id (INT)
national_id (VARCHAR(30))
BROKER
PK broker_id (INT)
broker_name (VARCHAR(100))
organization_name (VARCHAR(100))
phone (VARCHAR(15))
email (VARCHAR(100))
COVER_TYPE
PK cover_id (INT)
cover_name (VARCHAR(100))
PREMIUM_RATE
PK rate_id (INT)
FK category_id (INT)
FK cover_id (INT)
premium_amount (DECIMAL(10,2))
VEHICLE_MAKE
PK make_id (INT)
make_name (VARCHAR(100))
status (VARCHAR(10))
VEHICLE_MODEL
PK model_id (INT)
model_name (VARCHAR(100))
FK make_id (INT)
VEHICLE_CATEGORY
PK category_id (INT)
category_name (VARCHAR(50))
VEHICLE_BODY
PK body_id (INT)
body_type (VARCHAR(50))
VEHICLE_COLOR
PK color_id (INT)
color_name (VARCHAR(50))
VEHICLE
PK vehicle_id (INT)
FK customer_id (INT)
FK make_id (INT)
FK model_id (INT)
FK category_id (INT)
FK body_id (INT)
FK color_id (INT)
registration_no (VARCHAR(20))
manufacture_year (INT)
engine_no (VARCHAR(50))
chassis_no (VARCHAR(50))
QUOTE
PK quote_id (INT)
FK customer_id (INT)
FK vehicle_id (INT)
FK cover_id (INT)
quote_date (DATE)
basic_premium (DECIMAL(10,2))
tax (DECIMAL(10,2))
total_premium (DECIMAL(10,2))
status (VARCHAR(20))
QUOTE_COVER
PK quote_cover_id (INT)
FK quote_id (INT)
FK addon_id (INT)
ADDON_COVER
PK addon_id (INT)
addon_name (VARCHAR(100))
addon_amount (DECIMAL(10,2))
POLICY_SCHEDULE
PK schedule_id (INT)
FK policy_id (INT)
schedule_details (VARCHAR(255))
PAYMENT
PK payment_id (INT)
FK quote_id (INT)
payment_date (DATE)
payment_mode (VARCHAR(30))
currency (VARCHAR(20))
amount (DECIMAL(10,2))
payment_status (VARCHAR(20))
POLICY
PK policy_id (INT)
FK quote_id (INT)
FK payment_id (INT)
policy_number (VARCHAR(50))
issue_date (DATE)
expiry_date (DATE)
policy_status (VARCHAR(20))
DEBIT_CREDIT_NOTE
PK note_id (INT)
FK policy_id (INT)
note_type (VARCHAR(20))
amount (DECIMAL(10,2))
note_date (DATE)
BROKER_COMMISSION
PK commission_id (INT)
FK broker_id (INT)
FK policy_id (INT)
commission_amount (DECIMAL(10,2))
LEGEND
User & Security
Geography
Customer & Broker
Vehicle Master
Insurance Products
Transactions
Policy & Financial
One
Many
PK Primary Key
FK Foreign Key

Coding:

```
CREATE DATABASE motor_insurance_rr;
USE motor_insurance_rr;

-- user role
CREATE TABLE user_role (
    role_id INT PRIMARY KEY AUTO_INCREMENT,
    role_name VARCHAR(50) NOT NULL,
    status VARCHAR(10) DEFAULT 'Active'
);

-- login_user
CREATE TABLE login_user (
    login_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role_id INT,
    status VARCHAR(10) DEFAULT 'Active',
    FOREIGN KEY (role_id) REFERENCES user_role(role_id)
);
-- country
CREATE TABLE country (
    country_id INT PRIMARY KEY AUTO_INCREMENT,
    country_name VARCHAR(100) NOT NULL
)
-- state
CREATE TABLE state (
    state_id INT PRIMARY KEY AUTO_INCREMENT,
    state_name VARCHAR(100) NOT NULL,
```

```
    country_id INT,
    FOREIGN KEY (country_id) REFERENCES country(country_id)
);

--  city
CREATE TABLE city (
    city_id INT PRIMARY KEY AUTO_INCREMENT,
    city_name VARCHAR(100) NOT NULL,
    state_id INT,
    FOREIGN KEY (state_id) REFERENCES state(state_id)
);

-- customer
CREATE TABLE customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    gender VARCHAR(10),
    dob DATE,
    email VARCHAR(100),
    mobile VARCHAR(15),
    address VARCHAR(200),
    city_id INT,
    state_id INT,
    country_id INT,
    national_id VARCHAR(30),
    FOREIGN KEY(city_id) REFERENCES city(city_id),
    FOREIGN KEY(state_id) REFERENCES state(state_id),
    FOREIGN KEY(country_id) REFERENCES country(country_id)
);
```

```
--  broker
CREATE TABLE broker (
    broker_id INT PRIMARY KEY AUTO_INCREMENT,
    broker_name VARCHAR(100),
    organization_name VARCHAR(100),
    phone VARCHAR(15),
    email VARCHAR(100)
);

-- vehicle_make
CREATE TABLE vehicle_make (
    make_id INT PRIMARY KEY AUTO_INCREMENT,
    make_name VARCHAR(100),
    status VARCHAR(10)
);
-- vehicle_model
CREATE TABLE vehicle_model (
    model_id INT PRIMARY KEY AUTO_INCREMENT,
    model_name VARCHAR(100),
    make_id INT,
    FOREIGN KEY(make_id) REFERENCES vehicle_make(make_id)
);




-- vehicle_category
CREATE TABLE vehicle_category (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(50)
);
```

```
-- vehicle_color
CREATE TABLE vehicle_color (
    color_id INT PRIMARY KEY AUTO_INCREMENT,
    color_name VARCHAR(50) NOT NULL
);
-- vehicle_body
CREATE TABLE vehicle_body (
    body_id INT PRIMARY KEY AUTO_INCREMENT,
    body_type VARCHAR(50) NOT NULL
);

--  cover_type
CREATE TABLE cover_type (
    cover_id INT PRIMARY KEY AUTO_INCREMENT,
    cover_name VARCHAR(100) NOT NULL
);
-- addon_cover
CREATE TABLE addon_cover (
    addon_id INT PRIMARY KEY AUTO_INCREMENT,
    addon_name VARCHAR(100) NOT NULL,
    addon_amount DECIMAL(10,2)
);



-- vehicle
CREATE TABLE vehicle (
    vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    make_id INT,
    model_id INT,
    category_id INT,
```

```
    body_id INT,
    color_id INT,
    registration_no VARCHAR(20) UNIQUE,
    manufacture_year INT,
    engine_no VARCHAR(50),
    chassis_no VARCHAR(50),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (make_id) REFERENCES vehicle_make(make_id),
    FOREIGN KEY (model_id) REFERENCES vehicle_model(model_id),
    FOREIGN KEY (category_id) REFERENCES vehicle_category(category_id),
    FOREIGN KEY (body_id) REFERENCES vehicle_body(body_id),
    FOREIGN KEY (color_id) REFERENCES vehicle_color(color_id)
);

CREATE TABLE premium_rate (
    rate_id INT PRIMARY KEY AUTO_INCREMENT,
    category_id INT,
    cover_id INT,
    premium_amount DECIMAL(10,2),
    FOREIGN KEY (category_id) REFERENCES vehicle_category(category_id),
    FOREIGN KEY (cover_id) REFERENCES cover_type(cover_id)
);


CREATE TABLE quote (
    quote_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    vehicle_id INT,
    cover_id INT,
    quote_date DATE,
    basic_premium DECIMAL(10,2),
```

```
    tax DECIMAL(10,2),
    total_premium DECIMAL(10,2),
    status VARCHAR(20),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (vehicle_id) REFERENCES vehicle(vehicle_id),
    FOREIGN KEY (cover_id) REFERENCES cover_type(cover_id)
);

CREATE TABLE quote_cover (
    quote_cover_id INT PRIMARY KEY AUTO_INCREMENT,
    quote_id INT,
    addon_id INT,
    FOREIGN KEY (quote_id) REFERENCES quote(quote_id),
    FOREIGN KEY (addon_id) REFERENCES addon_cover(addon_id)
);

CREATE TABLE payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    quote_id INT,
    payment_date DATE,
    payment_mode VARCHAR(30),
    currency VARCHAR(20),


    amount DECIMAL(10,2),
    payment_status VARCHAR(20),
    FOREIGN KEY (quote_id) REFERENCES quote(quote_id)
);
```

```
CREATE TABLE policy (
    policy_id INT PRIMARY KEY AUTO_INCREMENT,
    quote_id INT,
    payment_id INT,
    policy_number VARCHAR(50) UNIQUE,
    issue_date DATE,
    expiry_date DATE,
    policy_status VARCHAR(20),
    FOREIGN KEY (quote_id) REFERENCES quote(quote_id),
    FOREIGN KEY (payment_id) REFERENCES payment(payment_id)
);

CREATE TABLE policy_schedule (
    schedule_id INT PRIMARY KEY AUTO_INCREMENT,
    policy_id INT,
    schedule_details VARCHAR(255),
    FOREIGN KEY (policy_id) REFERENCES policy(policy_id)
);

CREATE TABLE debit_credit_note (
    note_id INT PRIMARY KEY AUTO_INCREMENT,
    policy_id INT,
    note_type VARCHAR(20),
    amount DECIMAL(10,2),



    note_date DATE,
    FOREIGN KEY (policy_id) REFERENCES policy(policy_id)
);

CREATE TABLE broker_commission (
```

```
    commission_id INT PRIMARY KEY AUTO_INCREMENT,
    broker_id INT,
    policy_id INT,
    commission_amount DECIMAL(10,2),
    FOREIGN KEY (broker_id) REFERENCES broker(broker_id),
    FOREIGN KEY (policy_id) REFERENCES policy(policy_id)
);

INSERT INTO user_role(role_name,status) VALUES
('Admin','Active'),
('UnderWriter','Active'),
('Operational User','Active'),
('Broker','Active'),
('Sales Agent','Active');

INSERT INTO country(country_name) VALUES
('India'),
('UAE'),
('Singapore'),
('Malaysia'),
('Qatar');

INSERT INTO state(state_name,country_id) VALUES
('Tamil Nadu',1),
('Karnataka',1),
('Kerala',1),
('Andhra Pradesh',1),
('Telangana',1);
```

```
INSERT INTO city(city_name,state_id) VALUES
('Chennai',1),
('Coimbatore',1),
('Madurai',1),
('Bengaluru',2),
('Kochi',3);

INSERT INTO vehicle_make(make_name,status) VALUES
('Toyota','Active'),
('Hyundai','Active'),
('Honda','Active'),
('Tata','Active'),
('Mahindra','Active');

INSERT INTO vehicle_model(model_name,make_id) VALUES
('Innova',1),
('Fortuner',1),
('Creta',2),
('City',3),
('Nexon',4);

INSERT INTO vehicle_category(category_name) VALUES
('Private'),
('Commercial'),
('Taxi'),
('Motorcycle'),
('Sports');
```

```
INSERT INTO vehicle_color(color_name) VALUES
('White'),
('Black'),
('Silver'),
('Red'),
('Blue');

INSERT INTO vehicle_body(body_type) VALUES
('Sedan'),
('SUV'),
('Hatchback'),
('Van'),
('Truck');

INSERT INTO cover_type(cover_name) VALUES
('Comprehensive'),
('Third Party Liability'),
('Standalone Liability');

INSERT INTO addon_cover(addon_name,addon_amount) VALUES
('Zero Depreciation',2500),
('Roadside Assistance',1200),
('Engine Protection',3000),
('Key Replacement',800),
('Consumables Cover',1500);

INSERT INTO customer
(first_name,last_name,gender,dob,email,mobile,address,city_id,state_id,country_id,national_id)
VALUES
('Arun','Kumar','Male','1998-05-10','arun@gmail.com','9876543210','Chennai',1,1,1,'TN1001'),
('Priya','R','Female','1999-08-12','priya@gmail.com','9876543211','Coimbatore',2,1,1,'TN1002'),
```

('Karthik','S','Male','1997-03-15','karthik@gmail.com','9876543212','Madurai',3,1,1,'TN1003'),

('Divya','M','Female','2000-06-20','divya@gmail.com','9876543213','Chennai',1,1,1,'TN1004'),

('Vijay','K','Male','1996-09-25','vijay@gmail.com','9876543214','Bengaluru',4,2,1,'KA1005'),

('Anitha','P','Female','1998-12-18','anitha@gmail.com','9876543215','Kochi',5,3,1,'KL1006'),

('Rahul','D','Male','1997-01-30','rahul@gmail.com','9876543216','Chennai',1,1,1,'TN1007'),

('Meena','L','Female','1999-04-11','meena@gmail.com','9876543217','Coimbatore',2,1,1,'TN1008'),

('Suresh','B','Male','1995-07-22','suresh@gmail.com','9876543218','Madurai',3,1,1,'TN1009'),

('Kavya','N','Female','2001-10-05','kavya@gmail.com','9876543219','Chennai',1,1,1,'TN1010'),

('Hari','V','Male','1998-11-16','hari@gmail.com','9876543220','Bengaluru',4,2,1,'KA1011'),

('Nisha','A','Female','1997-02-08','nisha@gmail.com','9876543221','Kochi',5,3,1,'KL1012'),

('Praveen','C','Male','1996-08-09','praveen@gmail.com','9876543222','Chennai',1,1,1,'TN1013'),

('Deepa','T','Female','1999-01-17','deepa@gmail.com','9876543223','Coimbatore',2,1,1,'TN1014'),

('Ajith','R','Male','1998-03-23','ajith@gmail.com','9876543224','Madurai',3,1,1,'TN1015'),

('Swathi','K','Female','2000-07-27','swathi@gmail.com','9876543225','Chennai',1,1,1,'TN1016'),

('Manoj','P','Male','1995-12-14','manoj@gmail.com','9876543226','Bengaluru',4,2,1,'KA1017'),

('Revathi','S','Female','1998-05-28','revathi@gmail.com','9876543227','Kochi',5,3,1,'KL1018'),

('Ramesh','G','Male','1996-09-19','ramesh@gmail.com','9876543228','Chennai',1,1,1,'TN1019'),

('Keerthana','J','Female','1999-06-
03','keerthana@gmail.com','9876543229','Coimbatore',2,1,1,'TN1020'),

('Sanjay','M','Male','1997-04-04','sanjay@gmail.com','9876543230','Madurai',3,1,1,'TN1021'),

('Aarthi','V','Female','2001-11-13','aarthi@gmail.com','9876543231','Chennai',1,1,1,'TN1022'),

('Lokesh','H','Male','1998-02-21','lokesh@gmail.com','9876543232','Bengaluru',4,2,1,'KA1023'),

('Harini','R','Female','1997-07-07','harini@gmail.com','9876543233','Kochi',5,3,1,'KL1024'),

('Gokul','E','Male','1996-10-30','gokul@gmail.com','9876543234','Chennai',1,1,1,'TN1025');

INSERT INTO broker (broker_name, organization_name, phone, email) VALUES

('Arun Broker','ABC Insurance','9000000001','broker1@gmail.com'),

('Bala Broker','XYZ Insurance','9000000002','broker2@gmail.com'),

('Charles Broker','Safe Life','9000000003','broker3@gmail.com'),

('David Broker','Secure Insure','9000000004','broker4@gmail.com'),

('Elango Broker','Royal Insurance','9000000005','broker5@gmail.com'),

('Farooq Broker','Prime Insurance','9000000006','broker6@gmail.com'),

('Ganesh Broker','Future Secure','9000000007','broker7@gmail.com'),

('Hari Broker','Shield Insurance','9000000008','broker8@gmail.com'),

('Imran Broker','Trust Insurance','9000000009','broker9@gmail.com'),

('John Broker','Care Insurance','9000000010','broker10@gmail.com'),

('Karthik Broker','Fast Cover','9000000011','broker11@gmail.com'),

('Lokesh Broker','Best Policy','9000000012','broker12@gmail.com'),

('Manoj Broker','Life Cover','9000000013','broker13@gmail.com'),

('Naveen Broker','Safe Drive','9000000014','broker14@gmail.com'),

('Prakash Broker','Motor Protect','9000000015','broker15@gmail.com'),

('Ramesh Broker','National Insurance','9000000016','broker16@gmail.com'),

('Saravanan Broker','Secure Drive','9000000017','broker17@gmail.com'),

('Thiru Broker','Premium Cover','9000000018','broker18@gmail.com'),

('Uday Broker','Elite Insurance','9000000019','broker19@gmail.com'),

('Vignesh Broker','Auto Secure','9000000020','broker20@gmail.com'),

('Yogesh Broker','Trust Motor','9000000021','broker21@gmail.com'),

('Zakir Broker','Policy Hub','9000000022','broker22@gmail.com'),

('Ajay Broker','Road Shield','9000000023','broker23@gmail.com'),

('Bharath Broker','Drive Safe','9000000024','broker24@gmail.com'),

('Kiran Broker','Global Insurance','9000000025','broker25@gmail.com');

INSERT INTO login_user (username,password,role_id,status) VALUES

('admin1','admin@123',1,'Active'),

('uw1','uw@123',2,'Active'),

('op1','op@123',3,'Active'),

('broker1','broker@123',4,'Active'),

('agent1','agent@123',5,'Active'),

('admin2','admin@123',1,'Active'),

('uw2','uw@123',2,'Active'),

('op2','op@123',3,'Active'),

('broker2','broker@123',4,'Active'),

('agent2','agent@123',5,'Active'),

('admin3','admin@123',1,'Active'),

('uw3','uw@123',2,'Active'),

('op3','op@123',3,'Active'),

('broker3','broker@123',4,'Active'),

('agent3','agent@123',5,'Active'),

('admin4','admin@123',1,'Active'),

('uw4','uw@123',2,'Active'),

('op4','op@123',3,'Active'),

('broker4','broker@123',4,'Active'),

('agent4','agent@123',5,'Active'),

('admin5','admin@123',1,'Active'),

('uw5','uw@123',2,'Active'),

('op5','op@123',3,'Active'),

('broker5','broker@123',4,'Active'),

('agent5','agent@123',5,'Active');

INSERT INTO vehicle

(customer_id,make_id,model_id,category_id,body_id,color_id,registration_no,manufacture_year,engine_no,chassis_no)

VALUES

(1,1,1,1,2,1,'TN01AB1001',2022,'ENG1001','CHS1001'),

(2,2,3,1,2,2,'TN02AB1002',2023,'ENG1002','CHS1002'),

(3,3,4,1,1,3,'TN03AB1003',2021,'ENG1003','CHS1003'),

(4,4,5,1,3,4,'TN04AB1004',2022,'ENG1004','CHS1004'),

(5,5,5,2,2,5,'KA01AB1005',2023,'ENG1005','CHS1005'),

(6,1,2,1,2,1,'KL01AB1006',2020,'ENG1006','CHS1006'),

(7,2,3,1,2,2,'TN07AB1007',2021,'ENG1007','CHS1007'),

(8,3,4,1,1,3,'TN08AB1008',2022,'ENG1008','CHS1008'),

(9,4,5,2,4,4,'TN09AB1009',2023,'ENG1009','CHS1009'),

(10,5,5,1,2,5,'TN10AB1010',2021,'ENG1010','CHS1010'),

(11,1,1,1,2,1,'KA11AB1011',2022,'ENG1011','CHS1011'),

(12,2,3,1,2,2,'KL12AB1012',2023,'ENG1012','CHS1012'),

(13,3,4,1,1,3,'TN13AB1013',2021,'ENG1013','CHS1013'),

(14,4,5,1,3,4,'TN14AB1014',2022,'ENG1014','CHS1014'),

(15,5,5,2,2,5,'TN15AB1015',2023,'ENG1015','CHS1015'),

(16,1,2,1,2,1,'TN16AB1016',2022,'ENG1016','CHS1016'),

(17,2,3,1,2,2,'KA17AB1017',2021,'ENG1017','CHS1017'),

(18,3,4,1,1,3,'KL18AB1018',2022,'ENG1018','CHS1018'),

(19,4,5,2,4,4,'TN19AB1019',2023,'ENG1019','CHS1019'),

(20,5,5,1,2,5,'TN20AB1020',2021,'ENG1020','CHS1020'),

(21,1,1,1,2,1,'TN21AB1021',2022,'ENG1021','CHS1021'),

(22,2,3,1,2,2,'TN22AB1022',2023,'ENG1022','CHS1022'),

(23,3,4,1,1,3,'KA23AB1023',2021,'ENG1023','CHS1023'),

(24,4,5,1,3,4,'KL24AB1024',2022,'ENG1024','CHS1024'),

(25,5,5,2,2,5,'TN25AB1025',2023,'ENG1025','CHS1025');

INSERT INTO premium_rate (category_id, cover_id, premium_amount) VALUES

(1,1,5000),(1,2,3500),(1,3,6000),

(2,1,2000),(2,2,1500),(2,3,2500),

(3,1,8000),(3,2,6500),(3,3,9000),

(1,1,5200),(1,2,3600),(1,3,6100),

(2,1,2100),(2,2,1600),(2,3,2600),

(3,1,8100),(3,2,6600),(3,3,9100),

(1,1,5300),(1,2,3700),(1,3,6200),

(2,1,2200),(2,2,1700),(2,3,2700),

(3,1,8200),(3,2,6700);

```
INSERT INTO quote
(customer_id,vehicle_id,cover_id,quote_date,basic_premium,tax,total_premium,status)
VALUES
(1,1,1,'2026-07-01',5000,900,5900,'Pending'),
(2,2,2,'2026-07-01',4000,720,4720,'Approved'),
(3,3,1,'2026-07-01',4500,810,5310,'Pending'),
(4,4,3,'2026-07-02',8000,1440,9440,'Approved'),
(5,5,2,'2026-07-02',7000,1260,8260,'Pending'),

(6,1,1,'2026-07-03',5200,936,6136,'Approved'),
(7,2,2,'2026-07-03',4100,738,4838,'Pending'),
(8,3,3,'2026-07-03',9000,1620,10620,'Approved'),
(9,4,1,'2026-07-04',4300,774,5074,'Pending'),
(10,5,2,'2026-07-04',7200,1296,8496,'Approved'),

(11,1,3,'2026-07-05',6000,1080,7080,'Pending'),
(12,2,1,'2026-07-05',4800,864,5664,'Approved'),
(13,3,2,'2026-07-05',3900,702,4602,'Pending'),
(14,4,3,'2026-07-05',9500,1710,11210,'Approved'),
(15,5,1,'2026-07-06',5100,918,6018,'Pending'),

(16,1,2,'2026-07-06',5300,954,6254,'Approved'),
(17,2,3,'2026-07-06',8700,1566,10266,'Pending'),
(18,3,1,'2026-07-06',4600,828,5428,'Approved'),
(19,4,2,'2026-07-06',7400,1332,8732,'Pending'),
(20,5,3,'2026-07-07',9200,1656,10856,'Approved'),

(21,1,1,'2026-07-07',5000,900,5900,'Pending'),
(22,2,2,'2026-07-07',4000,720,4720,'Approved'),
(23,3,3,'2026-07-07',8000,1440,9440,'Pending'),
```

```
(24,4,1,'2026-07-07',4500,810,5310,'Approved'),
(25,5,2,'2026-07-07',7000,1260,8260,'Pending');

INSERT INTO quote_cover (quote_id, addon_id) VALUES
(1,1),(1,2),(1,3),
(2,1),(2,2),
(3,2),(3,3),
(4,1),(4,2),(4,3),
(5,1),

(6,2),(6,3),
(7,1),(7,2),
(8,3),
(9,1),(9,2),
(10,2),(10,3),

(11,1),(11,2),
(12,3),
(13,1),(13,2),
(14,2),(14,3),
(15,1);

INSERT INTO payment
(quote_id,payment_date,payment_mode,currency,amount,payment_status)
VALUES
(1,'2026-07-01','UPI','INR',5900,'Success'),
(2,'2026-07-01','Card','INR',4720,'Success'),
(3,'2026-07-01','UPI','INR',5310,'Pending'),
(4,'2026-07-02','Net Banking','INR',9440,'Success'),
(5,'2026-07-02','Card','INR',8260,'Pending'),
```

(6,'2026-07-03','UPI','INR',6136,'Success'),

(7,'2026-07-03','Card','INR',4838,'Pending'),

(8,'2026-07-03','UPI','INR',10620,'Success'),

(9,'2026-07-04','Card','INR',5074,'Pending'),

(10,'2026-07-04','UPI','INR',8496,'Success'),

(11,'2026-07-05','Card','INR',7080,'Pending'),

(12,'2026-07-05','UPI','INR',5664,'Success'),

(13,'2026-07-05','Card','INR',4602,'Pending'),

(14,'2026-07-05','UPI','INR',11210,'Success'),

(15,'2026-07-06','Card','INR',6018,'Pending'),

(16,'2026-07-06','UPI','INR',6254,'Success'),

(17,'2026-07-06','Card','INR',10266,'Pending'),

(18,'2026-07-06','UPI','INR',5428,'Success'),

(19,'2026-07-06','Card','INR',8732,'Pending'),

(20,'2026-07-07','UPI','INR',10856,'Success'),

(21,'2026-07-07','Card','INR',5900,'Pending'),

(22,'2026-07-07','UPI','INR',4720,'Success'),

(23,'2026-07-07','Card','INR',9440,'Pending'),

(24,'2026-07-07','UPI','INR',5310,'Success'),

(25,'2026-07-07','Card','INR',8260,'Pending');

-- Insert a complete set of policies with explicit policy_id so downstream

-- FK inserts (policy_schedule, debit_credit_note, broker_commission)

-- can reference policy IDs 1..25 reliably.

```
INSERT INTO policy
(policy_id,quote_id,payment_id,policy_number,issue_date,expiry_date,policy_status)
VALUES
(1,1,1,'POL1001','2026-07-01','2027-07-01','Active'),
(2,2,2,'POL1002','2026-07-01','2027-07-01','Active'),
(3,3,3,'POL1003','2026-07-02','2027-07-02','Active'),
(4,4,4,'POL1004','2026-07-02','2027-07-02','Active'),
(5,5,5,'POL1005','2026-07-03','2027-07-03','Active'),
(6,6,6,'POL1006','2026-07-03','2027-07-03','Active'),
(7,7,7,'POL1007','2026-07-04','2027-07-04','Active'),
(8,8,8,'POL1008','2026-07-04','2027-07-04','Active'),
(9,9,9,'POL1009','2026-07-05','2027-07-05','Active'),
(10,10,10,'POL1010','2026-07-05','2027-07-05','Active'),
(11,11,11,'POL1011','2026-01-12','2027-01-11','Active'),
(12,12,12,'POL1012','2026-01-13','2027-01-12','Active'),
(13,13,13,'POL1013','2026-01-14','2027-01-13','Pending'),
(14,14,14,'POL1014','2026-01-15','2027-01-14','Active'),
(15,15,15,'POL1015','2026-01-16','2027-01-15','Active'),
(16,16,16,'POL1016','2026-01-17','2027-01-16','Active'),
(17,17,17,'POL1017','2026-01-18','2027-01-17','Pending'),
(18,18,18,'POL1018','2026-01-19','2027-01-18','Active'),
(19,19,19,'POL1019','2026-01-20','2027-01-19','Cancelled'),
(20,20,20,'POL1020','2026-01-21','2027-01-20','Active'),
(21,21,21,'POL1021','2026-01-22','2027-01-21','Active'),
(22,22,22,'POL1022','2026-01-23','2027-01-22','Active'),
(23,23,23,'POL1023','2026-01-24','2027-01-23','Pending'),

(24,24,24,'POL1024','2026-01-25','2027-01-24','Active'),
(25,25,25,'POL1025','2026-01-26','2027-01-25','Active');
```

```
-- policy_schedule insertion moved later (after all policy rows exist)
INSERT INTO debit_credit_note (policy_id,note_type,amount,note_date) VALUES
(1,'Credit',200,'2026-07-01'),
(2,'Debit',150,'2026-07-01'),
(3,'Credit',300,'2026-07-02'),
(4,'Debit',100,'2026-07-02'),
(5,'Credit',250,'2026-07-03'),

(6,'Debit',180,'2026-07-03'),
(7,'Credit',220,'2026-07-04'),
(8,'Debit',140,'2026-07-04'),
(9,'Credit',260,'2026-07-05'),
(10,'Debit',160,'2026-07-05'),

(18,'Debit',275,'2026-01-19'),
(12,'Debit',120,'2026-07-06'),
(13,'Credit',210,'2026-07-07'),
(14,'Debit',190,'2026-07-07'),
(15,'Credit',270,'2026-07-07'),

(16,'Debit',130,'2026-07-07'),
(17,'Credit',240,'2026-07-07'),
(18,'Debit',110,'2026-07-07'),
(19,'Credit',280,'2026-07-07'),

(20,'Debit',170,'2026-07-07'),
(21,'Credit',260,'2026-07-07'),
(22,'Debit',150,'2026-07-07'),
(23,'Credit',300,'2026-07-07'),
(24,'Debit',140,'2026-07-07'),
(25,'Credit',250,'2026-07-07');
```

```
INSERT INTO broker_commission (broker_id,policy_id,commission_amount) VALUES
(1,1,300),
(2,2,250),
(3,3,400),
(1,4,280),
(2,5,320),

(3,6,350),
(1,7,300),
(2,8,270),
(3,9,420),
(1,10,310),

(2,11,260),
(3,12,390),
(1,13,300),
(2,14,330),
(3,15,410),

(1,16,290),
(2,17,280),
(3,18,430),
(1,19,300),
(2,20,340),

(3,21,360),
(1,22,310),
(2,23,295),
(3,24,415),
(1,25,305);
```

```
-- Using Basic Queries
-- ALL Customers
SELECT * FROM customer;

-- ALL Vehicles
SELECT * FROM vehicle;

-- ALL Policies
SELECT * FROM policy;

-- Male Customers
SELECT * FROM customer
WHERE gender = 'Male';

-- Chennai Customers
SELECT * FROM customer
WHERE city_id = 1;

-- Display Quotes with Premium Greater Than 10000
SELECT *
FROM quote
WHERE total_premium > 10000;

-- Display Customers in Alphabetical Order by First Name
SELECT *
FROM customer
ORDER BY first_name ASC;

-- Display All Recent Policies
SELECT *
FROM policy
```

```
ORDER BY issue_date DESC;

-- JOIN Queries
-- Customer + Vehicle
SELECT
    c.customer_id,
    c.first_name,
    v.registration_no,
    v.manufacture_year
FROM customer c
INNER JOIN vehicle v
ON c.customer_id = v.customer_id;

-- Quote + Customer
SELECT
    q.quote_id,
    c.first_name,
    q.total_premium
FROM quote q
INNER JOIN customer c
ON q.customer_id = c.customer_id;

-- Policy + Payment
SELECT
    p.policy_number,
    pay.amount,
    pay.payment_mode
FROM policy p
INNER JOIN payment pay
ON p.payment_id = pay.payment_id;
```

```
-- GROUP BY
SELECT
category_id,
COUNT(*) AS TotalVehicles
FROM vehicle
GROUP BY category_id;

SELECT
cover_id,
AVG(total_premium) AS AveragePremium
FROM quote
GROUP BY cover_id;

-- Sub Query
SELECT *
FROM customer
WHERE customer_id IN
(
SELECT customer_id
FROM quote
WHERE total_premium > 15000
);

--- Customer + Policy Details View
CREATE VIEW vw_customer_policy AS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    p.policy_number,
    p.issue_date,
```

```
    p.expiry_date,
    p.policy_status
FROM customer c
INNER JOIN quote q



    ON c.customer_id = q.customer_id
INNER JOIN payment pay
    ON q.quote_id = pay.quote_id
INNER JOIN policy p
    ON pay.payment_id = p.payment_id;

--- Execute
    SELECT * FROM vw_customer_policy;

--- STORED PROCEDURE
DELIMITER $$
CREATE PROCEDURE GetCustomerPolicy(IN cid INT)
BEGIN
    SELECT
        c.first_name,
        p.policy_number,
        p.issue_date,
        p.expiry_date
    FROM customer c
    JOIN quote q
        ON c.customer_id=q.customer_id
    JOIN payment pay
        ON q.quote_id=pay.quote_id
    JOIN policy p
        ON pay.payment_id=p.payment_id
```

```
   WHERE c.customer_id=cid;
END $$
DELIMITER ;

CALL GetCustomerPolicy(1);

--- TRIGGER
DELIMITER $$

CREATE TRIGGER trg_payment_status
AFTER INSERT
ON payment
FOR EACH ROW
BEGIN

UPDATE policy
SET policy_status='Active'
WHERE payment_id=NEW.payment_id;

END $$
DELIMITER ;
DELIMITER $$

-- FUNCTION
CREATE FUNCTION CalculateGST(amount DECIMAL(10,2))
RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN
RETURN amount*0.18;
```

```
END $$
DELIMITER ;



SELECT CalculateGST(25000);

-- INDEX
-- Email Search
CREATE INDEX idx_customer_email
ON customer(email);

-- Mobile Search
CREATE INDEX idx_customer_mobile
ON customer(mobile);

-- Vehicle Registration
CREATE INDEX idx_vehicle_registration
ON vehicle(registration_no);

-- Transaction Tables Data
-- Quote
INSERT INTO quote
(customer_id,vehicle_id,cover_id,quote_date,basic_premium,tax,total_premium,status)
VALUES
(11,11,1,'2026-01-11',12500,2250,14750,'Approved'),
(12,12,2,'2026-01-12',8200,1476,9676,'Approved'),
(13,13,1,'2026-01-13',14200,2556,16756,'Pending'),
(14,14,1,'2026-01-14',11800,2124,13924,'Approved'),
(15,15,2,'2026-01-15',9800,1764,11564,'Approved'),
(16,16,1,'2026-01-16',15200,2736,17936,'Approved'),
(17,17,2,'2026-01-17',8600,1548,10148,'Pending'),
```

(18,18,1,'2026-01-18',13800,2484,16284,'Approved'),

(19,19,2,'2026-01-19',9100,1638,10738,'Rejected'),

(20,20,1,'2026-01-20',15500,2790,18290,'Approved'),

(21,21,1,'2026-01-21',13200,2376,15576,'Approved'),

(22,22,2,'2026-01-22',8400,1512,9912,'Approved'),

(23,23,1,'2026-01-23',14500,2610,17110,'Pending'),

(24,24,2,'2026-01-24',8900,1602,10502,'Approved'),

(25,25,1,'2026-01-25',16100,2898,18998,'Approved');

-- quote_cover

INSERT INTO quote_cover (quote_id,addon_id)

VALUES

(11,2),

(12,4),

(13,5),

(14,1),

(15,3),

(16,2),

(17,4),

(18,5),

(19,1),

(20,3),

(21,2),

(22,5),

(23,4),

(24,1),

(25,3);

```
INSERT INTO payment
(quote_id,payment_date,payment_mode,currency,amount,payment_status)
VALUES
(11,'2026-01-12','UPI','INR',14750,'Success'),
(12,'2026-01-13','Card','INR',9676,'Success'),


(13,'2026-01-14','Net Banking','INR',16756,'Pending'),
(14,'2026-01-15','UPI','INR',13924,'Success'),
(15,'2026-01-16','Cash','INR',11564,'Success'),
(16,'2026-01-17','Card','INR',17936,'Success'),
(17,'2026-01-18','UPI','INR',10148,'Pending'),
(18,'2026-01-19','Net Banking','INR',16284,'Success'),
(19,'2026-01-20','Card','INR',10738,'Rejected'),
(20,'2026-01-21','UPI','INR',18290,'Success'),
(21,'2026-01-22','Cash','INR',15576,'Success'),
(22,'2026-01-23','Card','INR',9912,'Success'),
(23,'2026-01-24','UPI','INR',17110,'Pending'),
(24,'2026-01-25','Net Banking','INR',10502,'Success'),
(25,'2026-01-26','Card','INR',18998,'Success');

INSERT INTO policy_schedule(policy_id,schedule_details)
VALUES
(11,'Comprehensive Annual Plan'),
(12,'Third Party Coverage'),
(13,'Premium Protection Plan'),
(14,'Standard Coverage'),
(15,'Commercial Vehicle Cover'),
(16,'SUV Comprehensive'),
(17,'Pending Approval Schedule'),
(18,'Premium Motor Policy'),
```

(19,'Cancelled Policy'),

(20,'Private Vehicle Cover'),

(21,'Gold Protection Plan'),

(22,'Standard Annual Cover'),

(23,'Pending Verification'),

(24,'Taxi Insurance Plan'),

(25,'Complete Motor Insurance');

INSERT INTO debit_credit_note

(policy_id,note_type,amount,note_date)

VALUES

(11,'Credit',500,'2026-01-12'),

(12,'Debit',250,'2026-01-13'),

(13,'Credit',600,'2026-01-14'),

(14,'Debit',350,'2026-01-15'),

(15,'Credit',450,'2026-01-16'),

(16,'Debit',300,'2026-01-17'),

(17,'Credit',700,'2026-01-18'),

(18,'Debit',275,'2026-01-19'),

(19,'Credit',550,'2026-01-20'),

(20,'Debit',325,'2026-01-21'),

(21,'Credit',650,'2026-01-22'),

(22,'Debit',400,'2026-01-23'),

(23,'Credit',800,'2026-01-24'),

(24,'Debit',375,'2026-01-25'),

(25,'Credit',900,'2026-01-26');

```
INSERT INTO broker_commission
(broker_id,policy_id,commission_amount)
VALUES
(11,11,1200),
(12,12,900),
(13,13,1500),
(14,14,1000),


(15,15,950),
(16,16,1600),
(17,17,850),
(18,18,1700),
(19,19,800),
(20,20,1800),
(21,21,1100),
(22,22,1200),
(23,23,1300),
(24,24,1000),
(25,25,1900);

-- SQL Queries
-- Customers
SELECT * FROM customer;

-- Female Customers
SELECT *
FROM customer
WHERE gender='Female';
```

```
-- Customers (ORDER BY)
SELECT *
FROM customer
ORDER BY first_name;

-- Vehicles Manufactured After 2022
SELECT *
FROM vehicle
WHERE manufacture_year>2022;

-- Approved Quotes
SELECT *
FROM quote
WHERE status='Approved';

--  Active Policies
SELECT *
FROM policy
WHERE policy_status='Active';

-- Premium Greater Than 15000
SELECT *
FROM quote
WHERE total_premium>15000;

-- Total Customers
SELECT COUNT(*) AS Total_Customers
FROM customer;
```

```
-- Average Premium
SELECT AVG(total_premium) AS Average_Premium
FROM quote;

-- Maximum Premium
SELECT MAX(total_premium) AS Highest_Premium
FROM quote;

-- Minimum Premium
SELECT MIN(total_premium) AS Lowest_Premium
FROM quote;

-- Total Premium Collection
SELECT SUM(amount) AS Total_Collection
FROM payment
WHERE payment_status='Success';

-- Customer + Vehicle
SELECT
c.customer_id,
c.first_name,
v.registration_no
FROM customer c
INNER JOIN vehicle v
ON c.customer_id=v.customer_id;

-- Customer + Quote
SELECT
c.first_name,
q.quote_id,
q.total_premium
```

```
FROM customer c
JOIN quote q
ON c.customer_id=q.customer_id;

-- Quote + Payment
SELECT
q.quote_id,
pay.amount,
pay.payment_mode
FROM quote q
JOIN payment pay
ON q.quote_id=pay.quote_id;

-- Payment + Policy
SELECT
pay.payment_id,
p.policy_number,
p.policy_status
FROM payment pay
JOIN policy p
ON pay.payment_id=p.payment_id;

-- Customer + Policy
SELECT
c.first_name,
p.policy_number
FROM customer c
JOIN quote q
ON c.customer_id=q.customer_id
JOIN payment pay
ON q.quote_id=pay.quote_id
```

```
JOIN policy p
ON pay.payment_id=p.payment_id;

-- Vehicle + Vehicle Make
SELECT
v.registration_no,
m.make_name
FROM vehicle v
JOIN vehicle_make m
ON v.make_id=m.make_id;

-- Vehicle + Model
SELECT
v.registration_no,
vm.model_name
FROM vehicle v
JOIN vehicle_model vm
ON v.model_id=vm.model_id;

-- Customer Count by State
SELECT
state_id,
COUNT(*) AS TotalCustomers
FROM customer
GROUP BY state_id;

-- LEFT JOIN
SELECT
c.first_name,
q.quote_id
FROM customer c
```

```
LEFT JOIN quote q
ON c.customer_id=q.customer_id;

--  RIGHT JOIN
SELECT
q.quote_id,
c.first_name
FROM customer c
RIGHT JOIN quote q
ON c.customer_id=q.customer_id;

-- CROSS JOIN
SELECT
c.first_name,
ct.cover_name
FROM customer c
CROSS JOIN cover_type ct;

-- SELF JOIN
SELECT
A.first_name,
B.first_name
FROM customer A
JOIN customer B
ON A.customer_id<>B.customer_id
LIMIT 10;
```

```
-- GROUP BY
SELECT
status,
COUNT(*) TotalQuotes
FROM quote
GROUP BY status;

-- HAVING
SELECT
status,
COUNT(*) TotalQuotes
FROM quote
GROUP BY status
HAVING COUNT(*)>2;

-- UNION
SELECT first_name FROM customer
UNION
SELECT broker_name FROM broker;

-- UNION ALL
SELECT first_name FROM customer
UNION ALL
SELECT broker_name FROM broker;

-- EXISTS
SELECT *
FROM customer c
WHERE EXISTS
(
SELECT 1
```

```
FROM quote q
WHERE c.customer_id=q.customer_id
);

-- NOT EXISTS
SELECT *
FROM customer c
WHERE NOT EXISTS
(
SELECT 1
FROM quote q
WHERE c.customer_id=q.customer_id
);

-- CASE
SELECT
quote_id,
total_premium,
CASE
WHEN total_premium>=15000 THEN 'High'
WHEN total_premium>=10000 THEN 'Medium'
ELSE 'Low'
END AS Premium_Level


FROM quote;

-- IN
SELECT *
FROM customer
WHERE city_id IN(1,2,3);
```

```
-- BETWEEN
SELECT *
FROM quote
WHERE total_premium
BETWEEN 10000 AND 18000;

-- LIKE
SELECT *
FROM customer
WHERE first_name LIKE 'A%';

-- DISTINCT
SELECT DISTINCT state_id
FROM customer;

-- Sub Query
SELECT *
FROM quote
WHERE total_premium=
(
SELECT MAX(total_premium)
FROM quote
);

-- Correlated Sub Query
SELECT *
FROM customer c
WHERE EXISTS
(
SELECT *
FROM vehicle v
```

```
WHERE c.customer_id=v.customer_id
);

-- Nested Query
SELECT *
FROM customer
WHERE customer_id
IN
(
SELECT customer_id
FROM quote
WHERE total_premium>
(
SELECT AVG(total_premium)
FROM quote
)
);

-- Customer Wise Premium
SELECT
customer_id,
SUM(total_premium)
FROM quote
GROUP BY customer_id;

-- State Wise Customers
SELECT
state_id,
COUNT(*)
FROM customer
GROUP BY state_id;
```

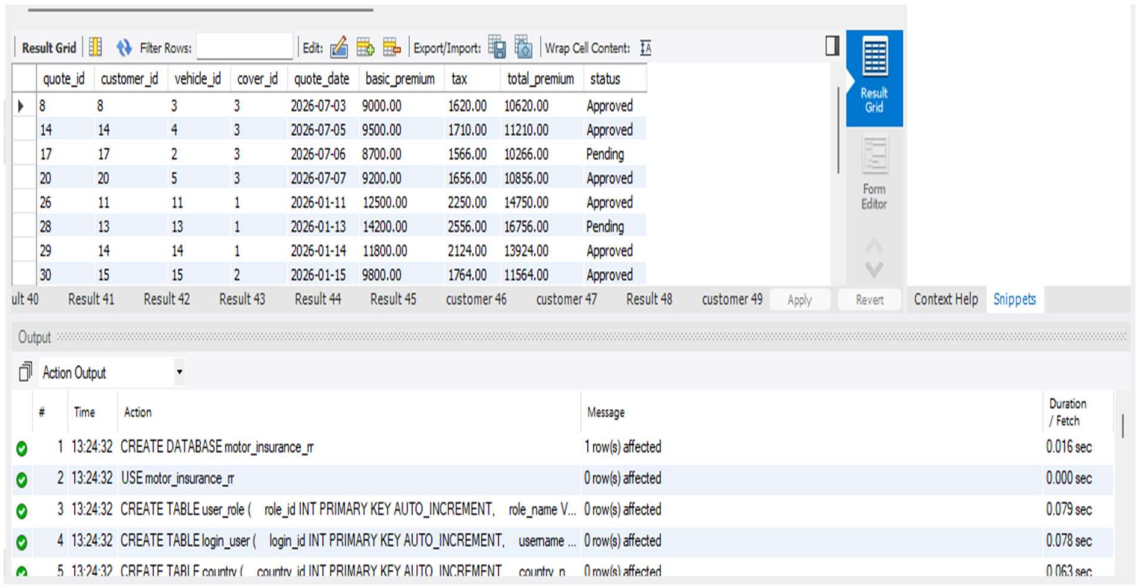
```
-- Payment Success
SELECT *
FROM payment
WHERE payment_status='Success';

-- Payment Pending
SELECT *
FROM payment
WHERE payment_status='Pending';

-- Policy Expiry
SELECT *
FROM policy
ORDER BY expiry_date;

-- Latest Policies
SELECT *
FROM policy
ORDER BY issue_date DESC;

-- Vehicle Count
SELECT
category_id,
COUNT(*)
FROM vehicle
GROUP BY category_id;

-- Broker Commission
SELECT
broker_id,
SUM(commission_amount)
```

```
FROM broker_commission
GROUP BY broker_id;

-- Highest Commission
SELECT MAX(commission_amount)
FROM broker_commission;

-- Lowest Commission
SELECT MIN(commission_amount)
FROM broker_commission;

-- Average Commission
SELECT AVG(commission_amount)
FROM broker_commission;

-- Total Commission
SELECT SUM(commission_amount)
FROM broker_commission;

-- Total Policies
SELECT COUNT(*)
FROM policy;

-- Total Vehicles
SELECT COUNT(*)
FROM vehicle;

-- Total Quotes
SELECT COUNT(*)
FROM quote;
```

```
-- Total Payments
SELECT COUNT(*)
FROM payment;

-- Customer Email Search
SELECT *
FROM customer
WHERE email='arun@gmail.com';

-- Vehicle Registration Search
SELECT *
FROM vehicle
WHERE registration_no='TN01AB1001';

-- Active Brokers
SELECT *
FROM broker;

-- Comprehensive Cover
SELECT *
FROM cover_type
WHERE cover_name='Comprehensive';

-- Third Party Cover
SELECT *
FROM cover_type
WHERE cover_name='Third Party Liability';
```

```
-- Dashboard Summary
SELECT
(SELECT COUNT(*) FROM customer) AS Customers,
(SELECT COUNT(*) FROM vehicle) AS Vehicles,
(SELECT COUNT(*) FROM quote) AS Quotes,
(SELECT COUNT(*) FROM payment) AS Payments,
(SELECT COUNT(*) FROM policy) AS Policies;

-- Final JOIN Test
SELECT
    c.first_name,
    v.registration_no,
    q.total_premium,
    p.policy_number
FROM customer c
JOIN vehicle v
    ON c.customer_id = v.customer_id
JOIN quote q
    ON c.customer_id = q.customer_id
JOIN payment pay
    ON q.quote_id = pay.quote_id
JOIN policy p
    ON pay.payment_id = p.payment_id;
```

SCREENSHORT:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

quote_id	customer_id	vehicle_id	cover_id	quote_date	basic_premium	tax	total_premium	status
8	8	3	3	2026-07-03	9000.00	1620.00	10620.00	Approved
14	14	4	3	2026-07-05	9500.00	1710.00	11210.00	Approved
17	17	2	3	2026-07-06	8700.00	1566.00	10266.00	Pending
20	20	5	3	2026-07-07	9200.00	1656.00	10856.00	Approved
26	11	11	1	2026-01-11	12500.00	2250.00	14750.00	Approved
28	13	13	1	2026-01-13	14200.00	2556.00	16756.00	Pending
29	14	14	1	2026-01-14	11800.00	2124.00	13924.00	Approved
30	15	15	2	2026-01-15	9800.00	1764.00	11564.00	Approved

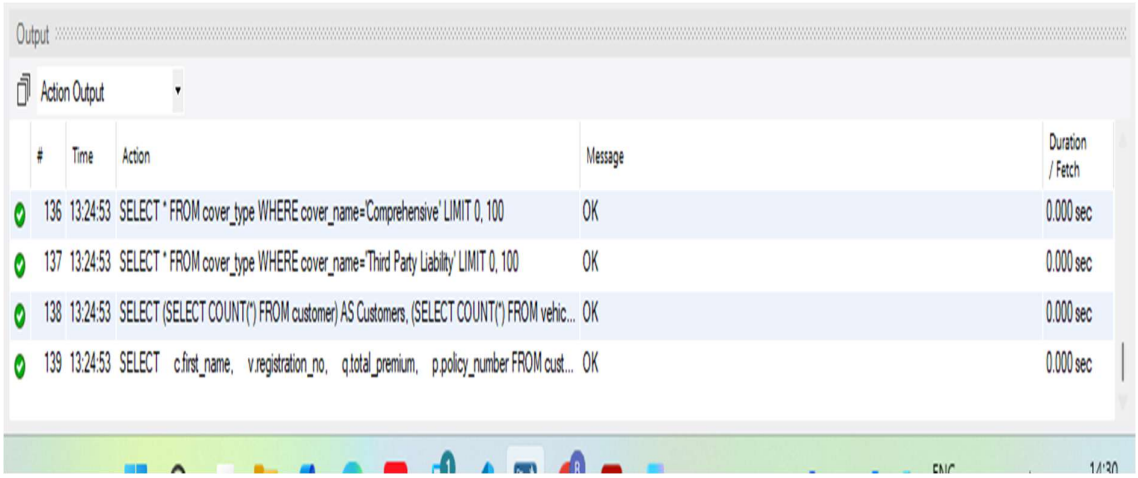
Result Grid | Form Editor

ult 40 | Result 41 | Result 42 | Result 43 | Result 44 | Result 45 | customer 46 | customer 47 | Result 48 | customer 49 | Apply | Revert | Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:24:32	CREATE DATABASE motor_insurance_rr	1 row(s) affected	0.016 sec
2	13:24:32	USE motor_insurance_rr	0 row(s) affected	0.000 sec
3	13:24:32	CREATE TABLE user_role (role_id INT PRIMARY KEY AUTO_INCREMENT, role_name V...	0 row(s) affected	0.079 sec
4	13:24:32	CREATE TABLE login_user (login_id INT PRIMARY KEY AUTO_INCREMENT, username ...	0 row(s) affected	0.078 sec
5	13:24:32	CREATE TABLE country (country_id INT PRIMARY KEY AUTO_INCREMENT, country_n...	0 row(s) affected	0.063 sec



Output

Action Output

#	Time	Action	Message	Duration / Fetch
136	13:24:53	SELECT * FROM cover_type WHERE cover_name='Comprehensive' LIMIT 0, 100	OK	0.000 sec
137	13:24:53	SELECT * FROM cover_type WHERE cover_name='Third Party Liability' LIMIT 0, 100	OK	0.000 sec
138	13:24:53	SELECT (SELECT COUNT(*) FROM customer) AS Customers, (SELECT COUNT(*) FROM vehic...	OK	0.000 sec
139	13:24:53	SELECT c.first_name, v.registration_no, q.total_premium, p.policy_number FROM cust...	OK	0.000 sec

12. Features

- Secure Login
- Customer Management
- Vehicle Registration
- Premium Calculation
- Quote Generation
- Policy Management
- Payment Management
- Claim Processing
- Report Generation

13. Advantages

- Easy to use
- Reduces paperwork
- Faster processing
- Secure database
- Easy report generation
- Better customer service
- Improved data accuracy

14. Limitations

- Requires database connectivity.
- Requires authorized users.
- Depends on system maintenance.

15. Future Enhancements

- Online Payment Gateway
- SMS Notifications
- Email Notifications
- Mobile Application
- AI-based Premium Prediction
- Online Claim Upload
- Dashboard with Reports

16. Testing

The system has been tested for:

- Login Validation
- Customer Registration
- Vehicle Registration
- Quote Generation
- Premium Calculation
- Policy Generation
- Payment Processing
- Claim Processing

17. Conclusion

The **Motor Insurance Management System** is an efficient database application that automates motor insurance operations. It manages customer, vehicle, quote, policy, payment, and claim information in a centralized database. The system improves efficiency, minimizes manual work, ensures data accuracy, and provides faster insurance services.

18. References

1. MySQL 8.0 Documentation
2. MySQL Workbench User Guide
3. Database Management System Concepts
4. Motor Insurance Business Requirements (Project Document)
5. SQL Language Reference